

(Chapter 5 | Replication)

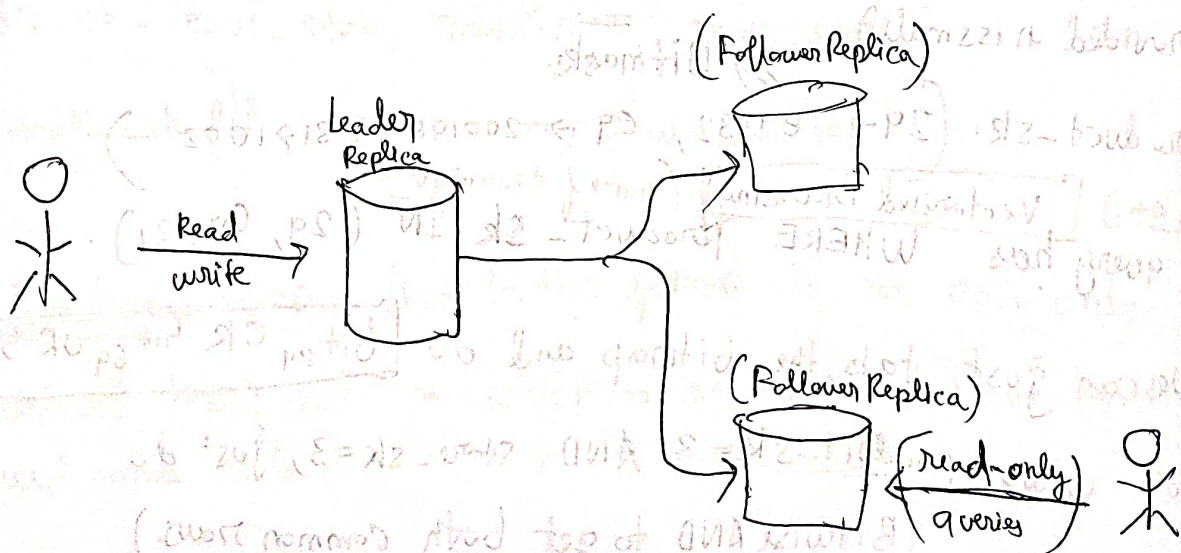
Why Replication?

- * Keep Data in same region or nearby regions to users
 - ↳ reduce latency
- * Availability → Some machine goes down, its replica can serve traffic in its place
- * Can serve more Read Throughput, As replicas can serve concurrent traffic

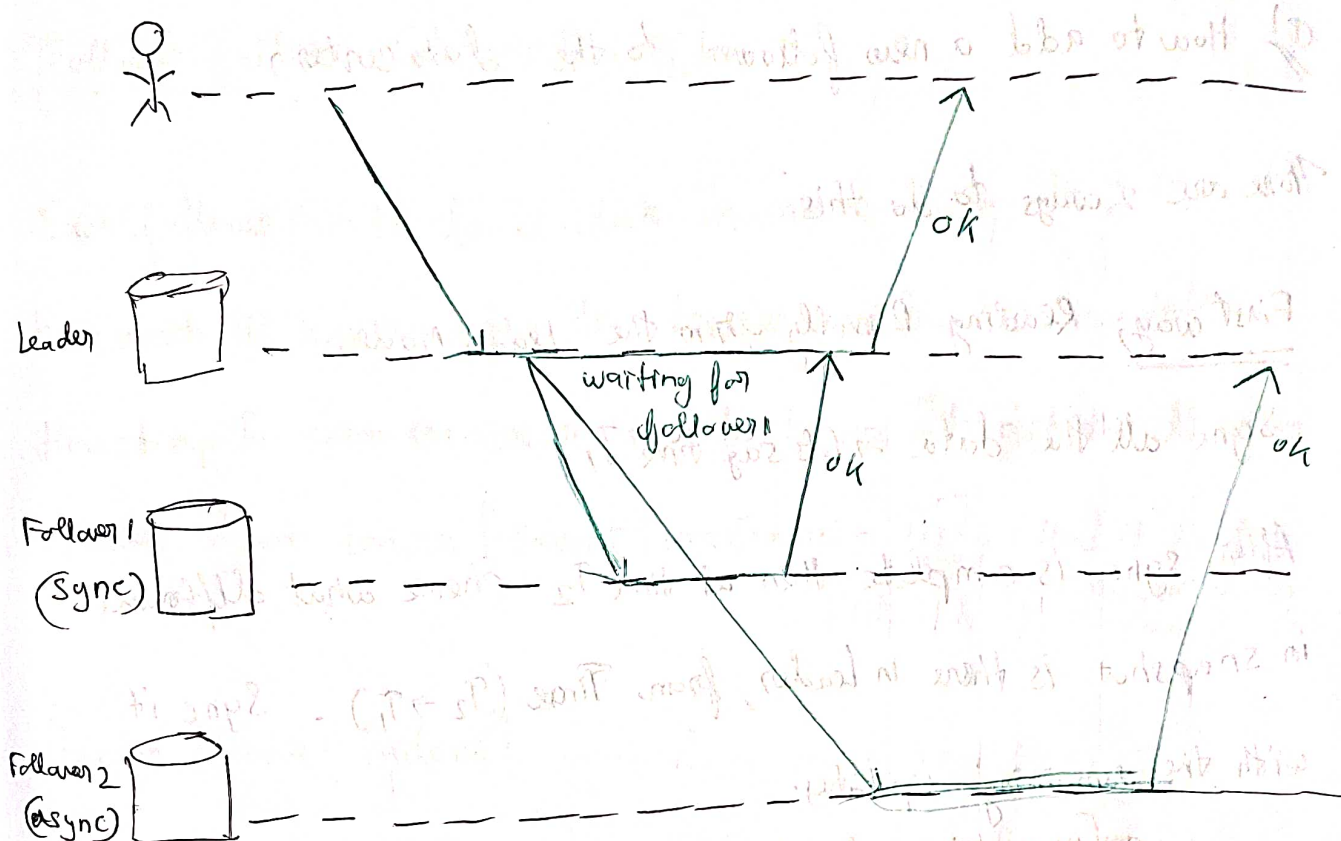
(How to do Replication)?

There is a master-slave architecture. Of multiple nodes, 1 node is assigned as the Leader and other nodes are replicas (followers).

Read requests are served by any of the nodes, (master/followers).



(async vs sync Process in Replication)



Sync Replication:

Advantage

- * Follower is guaranteed to have upto date copy - Leader only confirms when followers send ok.

Disadvantage

- * If follower takes lot of time, or goes into halt and does not respond, Leader cannot confirm write.

Async Replication:

Advantage

- * Leader can send ok to user and does not make system halt

Disadvantage

- * If completely async, some writes in Leader [if leader crashes] might not be propagated.

Setting up mix of async and sync $\rightarrow$ (Semi-Synchronous) process

Q) How to add a new follower to the data center?

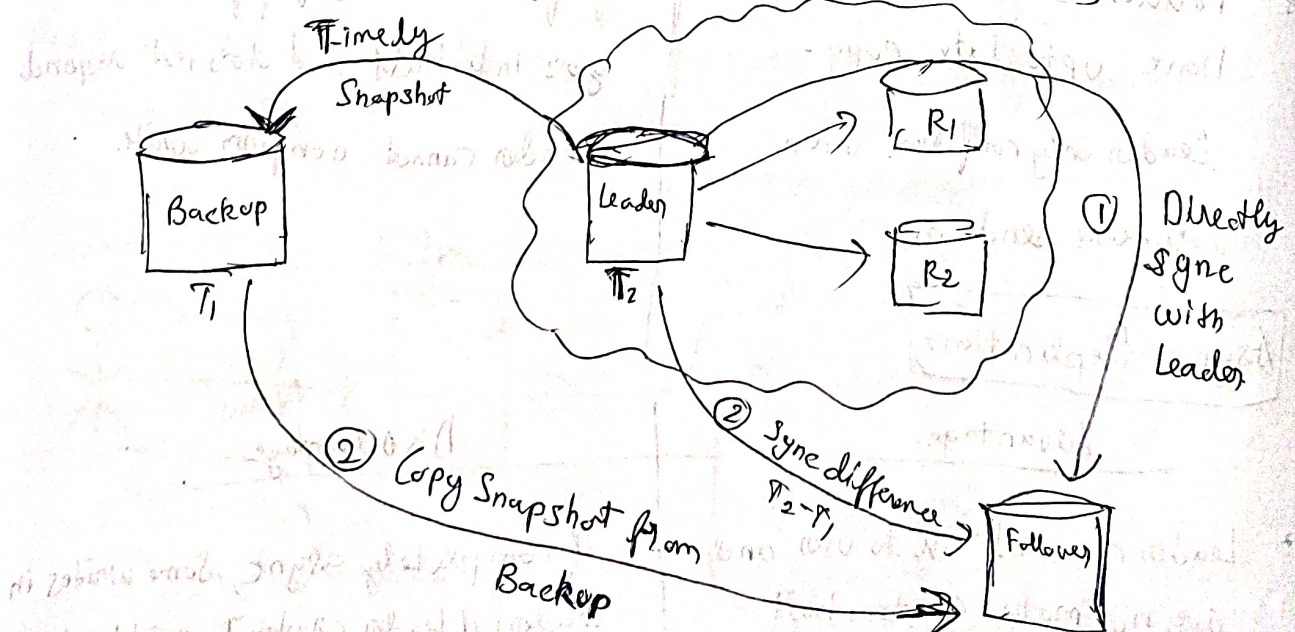
There are 2 ways to do this.

First way, Reading Directly from the Leader Node.

Sync all the data $\rightarrow$ let say time T_1

After sync is complete then at time T_2 check what difference in snapshot is there in leader, from Time ($T_2 \rightarrow T_1$). Sync it with the remaining updates.

2nd way $\rightarrow$ [method is preferred because load on leader is minimised].
The snapshot that is maintained as backup of the leader, we sync follower [newly created] with that, and after sync time T_2 ; we will sync with leader the differences ($T_2 \rightarrow T_1$)



Handling Node Outages

[Follower failure: Catch-up Recovery]

Each follower on its local disk maintains a log, that is, how much it has read from the Leader ~~and~~. Let's say till timestamp T_1 . When the server comes back up, ~~It~~ gets the db entry updates from Leader $[Current\ TimeStamp - T_1]$, (time it has the snapshot already written).

[Leader failure: Failover]

(Failover) $\rightarrow$ When the leader fails steps are:

1. One of followers is promoted to new leader
2. Clients need to reconfigure to route requests to new leader.
3. Other followers need to start consuming changes from new leader.

* Determining the leader has failed: Heartbeat messages are sent to nodes and communicate across each other. 30 seconds no response $\rightarrow$ dead.

* Choosing a new leader $\rightarrow$ Replica with most up-to-date updates.

* Rerouting client traffic to the newly elected Leader.

(Some Incidents with Followers)

Lets say the leader writes async to the followers

Before writing some of the updates to the replica, the leader failed.

Now the new leader [old follower] has some data missing.

Now in case the old leader [which crashed] comes UP, then at that time the (old leader) and the new leader [previously follower]

Conflict will be happening. (New leader) has some conflict till updates with (old leader)

Solution: Basic Solution is to discard the updates of the old leader which were supposed to be propagated asynchronously.

↗
[Github Incident]

The above solution, lead to a github incident. The MySQL dB had its leader node failed and new Replica was elected as Leader.

OLD Leader.

123 → org1

124 → org1

125 → org1

126 → org1

127 → org1

128 → org1

Primary key

was auto incremented.

New Leader (Replica initially)

123 → org1

124 → org1

125 → org1

(After this leader crashed and async updates did not happen)

New Writer.

123 $\rightarrow$ org1

124 $\rightarrow$ org1

125 $\rightarrow$ org1

126 $\rightarrow$ org2

127 $\rightarrow$ org2

128 $\rightarrow$ org2.

Now ~~the~~ old

Leader comes
back online.

old Leader.

123 $\rightarrow$ org1

124 $\rightarrow$ org1

125 $\rightarrow$ org1

126 $\rightarrow$ org1

127 $\rightarrow$ org1

128 $\rightarrow$ org1

These writes
are discarded as it did not get written asynchronously.

Now what happen is primary keys (126, 127, 128),
Github repos of primary key [126, 127, 128] which are of org1,
are now visible to org2 and org1 has lost access.

Basically repository of org1 is got migrated to org2, rather
access got migrated.

- * If two Nodes. Earlier Leader and New Leader, both act as
Leader, it causes conflicts $\rightarrow$ split brain.

- * Right time out need to set for failover.

Too small timeout $\rightarrow$ Unnecessary fail over for small network glitch.

Longer timeout $\rightarrow$ More time required for recovery

(Implementations of Replication logs)

(Statement-based Replication Log)

Leader logs every write request (statement) that it executes and sends it to followers $\rightarrow$ (INSERT, DELETE or UPDATE)

(Disadvantages of Statement based Replication Log)

- * Suppose any statement has non-deterministic function, such as NOW() $\rightarrow$ datetime, or RAND() $\rightarrow$ Random Number, can lead to replicas have different values and cause discrepancy.
- If statement includes auto incrementing column or they depend on populating the data based on some other column, then we need to make sure that all the write across all the replicas happen in the same order, and no data is modified before these writes are done completely.
- Triggers, if some statements have can lead to additional side effects.

Write-Ahead log (WAL) Shipping

In chapter 3 we saw LSM trees and B-trees using write ahead log. Basically before doing any modification, they ~~do it on~~ store it in disk in a log file.

WAL has a disadvantage, the problem it has is it stores exactly which bytes and which disk blocks were changed. Basically operating system level.

Hence it becomes very tightly coupled with the OS. Next day DB changes OS, all WALs are to be modified as per new OS.

Logical (Row-based) log replication.

Does not contain low level details, it contains just the details of the row.

③③③ For an inserted row, the log contains new values of all columns

③③ For deleted row, contains a unique identifier to identify the row

③③ For update contains primary key, and only the ~~rows~~ columns which need updating.

Since it's decoupled with DB storage engine internals, it can be kept more easily as backward compatible, allowing leader and follower to run two different versions of DB engine.

Trigger based Replication

This is a custom replication log written by developer as needed. Use triggers so that when leader writes, the triggers update in replicas.

Disadvantage: More overhead than other techniques and prone to bugs.

Problems with Replication Lag

We obviously don't know how much a replica can fall behind, because of replication lag. And Followers and leader might give inconsistent results $\rightarrow$ (Eventual Consistency).

(Reading Your Own Writes)

Facebook, Instagram: Similar applications like these let the user submit some data and then view what they have submitted.

For example, adding a comment, making a post.

We know writes happen on Leader, and reads on follower, if the read went to follower and follower has replication lag, user won't see their updated data.

To solve this we introduced [read-after write consistency]
also known as [read-your writes consistency]

Ways to achieve [read-your writes consistency]

(80) Simple rule is always when user wants to read their own data, direct the traffic to leader only [which will obviously have the latests], and if user wants to read somebody else's profile / data direct it to follower.

(80) Lets say a lot of details could be editable by the user, for example facebook profile. Then for each edit they are making, ~~it~~ for viewing it the request goes to leader always, Destroying the point of having replicas. Instead of directing all the traffic to leader, we can monitor replication lags on the replicas.

(20) The client knows the timestamp they have made the last update. Using this we can check the replica if it has update till this timestamp. Else we wait for replication lag to clear up.

☞ (Cross-device read after write consistency)

Mobile, Laptop, Another device updates. →

- Now client needs to maintain centralised last updated timestamp, As timestamp update of mobile will not be reflected on laptop.
- Very difficult to map traffic from different clients to same data center. Because we want to direct them to same leader / data center.

(Monotonic Reads)

Moving backward in time,

User uploaded a comment or edited a comment.

First time he reads, read request goes to replica 1, (up-to-date)

2nd time he reads, read request goes to replica 2 (not-up-to-date)

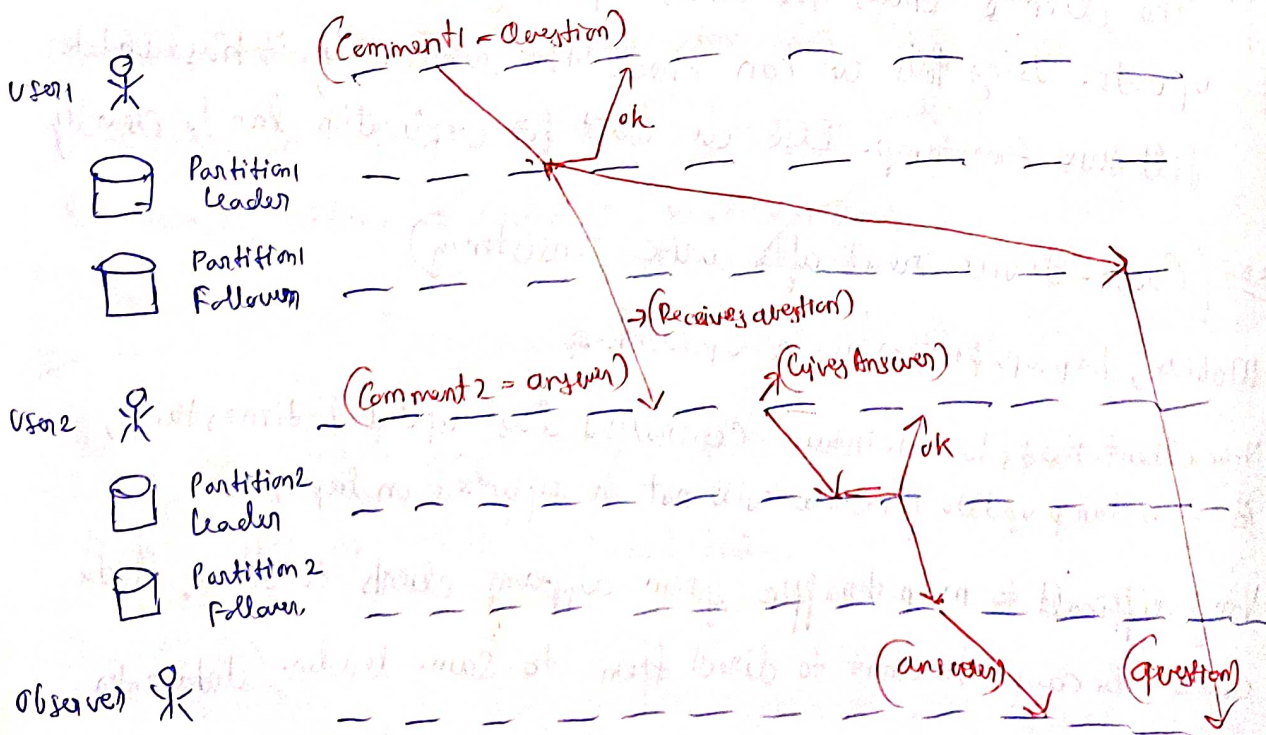
User will see old comment [unedited data].

Monotonic Reads, provide the guarantee that user will not see time go backwards. Once he has read a particular data, he will not read older data.

One way of guaranteeing this is by ~~that~~ [user $\rightarrow$ replica map] for read. Each user will read from particular replica only.

Consistent Prefix Reads

Let's say fight going on in Instagram comment section.



You can see how a third person will see the answer before the question in the comment section.

~~Consistent~~

consistent prefix reads $\rightarrow$ this guarantees if a user is reading then he will read, the writes in the sequence they were written.

(maybe some writes might be missed but sequence will never break)

(Chapter 6) $\rightarrow$ we will discuss partitioned (sharded database) tackling this problem.

$\hookrightarrow$ one ^(8.8.2) solution is to make sure that writes that are casually related to each other are written to same partition, to keep sequence.

Multi Leader Replication.

Major downside of single-leader replication, then there is only one leader and if for any network ~~int~~ interruption the leader fails we cannot write to the DB.

A natural extension is to allow more nodes to allow to accept writes - Replication still happens the same way.

We have master-master / Active-Active replication.

$\Downarrow$
Check Hussain Nasser Networking
Playlist

(Use Cases of Multi-Leader Replication)

With single leader-based replication out of many datacenters only 1 datacenter will have the leader node.

In multi-data center, each datacenter, within each data-center there is a leader node.

Performance: In single leader writing and routing traffic to ~~each~~ some data center will cause performance issue. In multi-leader each data center can process the request and replicate asynchronously in other data centers.

Tolerance of datacenter outages: No need to re-appoint leader in a multi-leader format when a data center fails.

Tolerance of Network Problems:

A single-leader configuration is very sensitive to problems because writes are made synchronously.

A multi-leader can avert network problems because it writes asynchronously.

Disadvantage of Multi-leader.

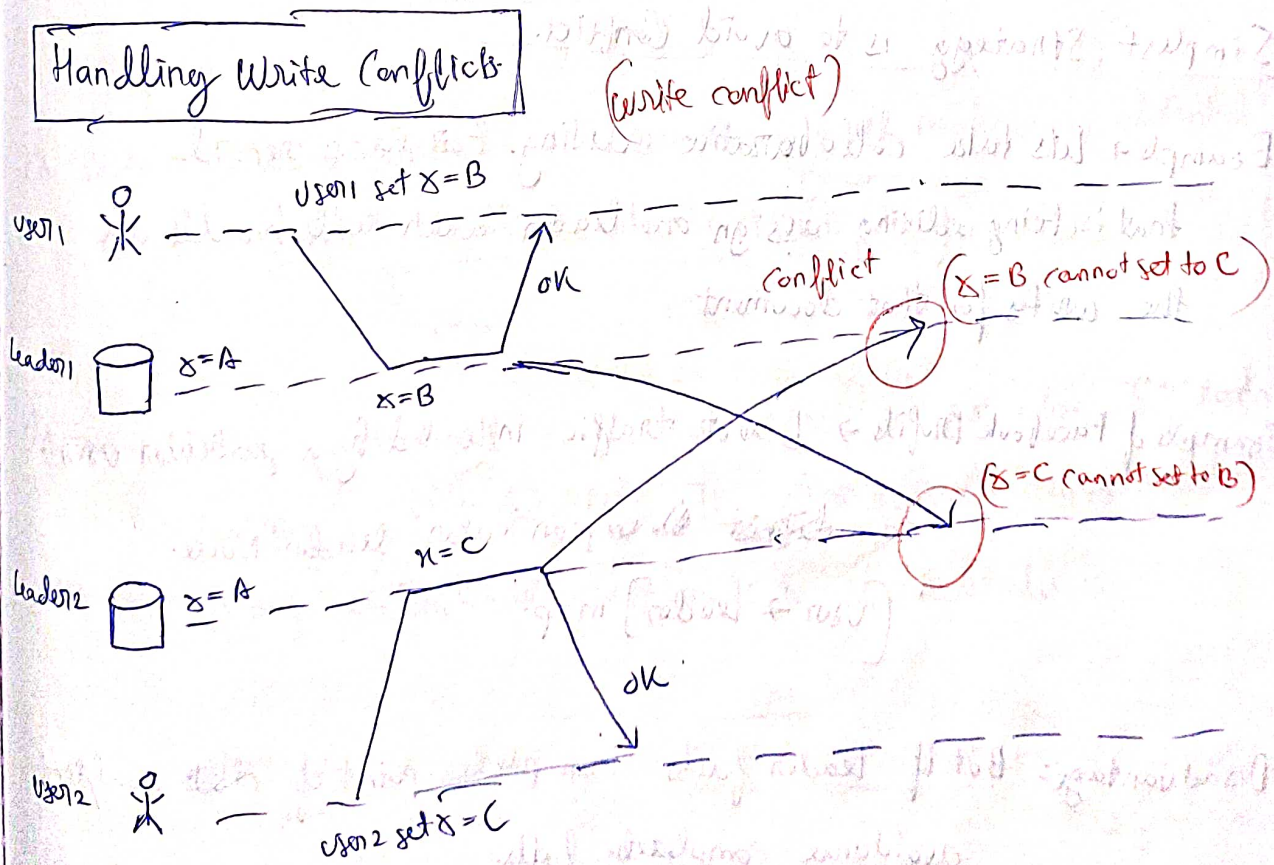
- * Auto-Incrementing keys, triggers and integrity constraints can be problematic.

(Some Examples)

Calendar App $\rightarrow$ When suppose internet is out and you want to update your calendar with a meeting or something. The devices with which you can access it mobile, laptop ~~these~~ these act as your local datacenters and when internet comes back, they write back to other data centers such as laptop; other devices

Collaborative Editing: Multiple users editing the same document.

Each users machine becomes the local data center.



Synchronous v/s Asynchronous Conflict Detection

As the figure we saw the write conflict, if we handle it asynchronously, but it might be too late to handle it then, as a lot of writes might already happened.

To handle the writes synchronously there are two options.

1. Only have 1 writer thread $\rightarrow$ But reduces performance

Also if we handle it synchronously, we would have to wait for each write to get completely replicated on each replica. But that destroyed the whole point of multi-leader.

Conflict Avoidance

Simplest Strategy is to avoid Conflict.

Example $\rightarrow$ Let's take collaborative editing. For each record that is being editing assign one leader which will handle all the writes for that document

Example of Facebook Profile $\rightarrow$ Direct traffic intended by a particular user to a ~~direct~~ ~~to~~ particular leader node
(user $\rightarrow$ leader) map

Disadvantage: But if leader fails complete point of ~~conflict~~ conflict avoidance completely fails

Converging towards a consistent State

A Single Leader applies writes to the database in the order they arrive so its not possible in multi leader.

Convergent Way

- * Give each write a unique ID (timestamp, UUID) pick the write with highest ID as winner and discard others. If timestamp is used this technique is called last write wins (LWW)

- * Write Application code and display user that there is a conflict and let user decide → [Github merge Conflict]

Custom Conflict Resolution Logic.

On write → During write from Replication log if conflict is detected
a conflict handler would be called.
↓
Custom.

On Read → During write all conflicts are stored in one place. For each key there is a list of conflicts. Then during each key is read, all the conflicts are displayed to user to resolve it.

Automatic Conflict Resolution

During Amazon shopping experience has a conflict resolution mechanism, that it would choose over removes item and preserve item, the preserve item option.

Hence during conflict resolution it is seen that customers removed an item from the cart but it was still there after refresh.

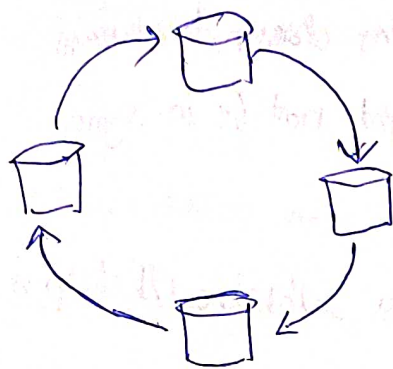
Conflict-free replicated datatypes $\rightarrow$ Concurrent Hashmap.

Mergeable persistent data structures $\rightarrow$ Merkle trees of Git/GitHub
GitHub maintains history to merge in version control

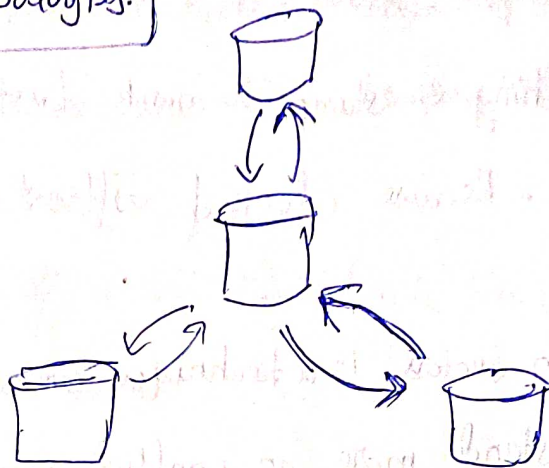
What is a conflict?

A Room booking application is there, Both users are trying to book the same room. In case of in memory datastore we can maintain a Hashmap-[Concurrent Hashmap], But in case of databases, two writes going to different nodes, how do you respond to users that their room is booked or not.

Multi-Leader Replication Topologies.

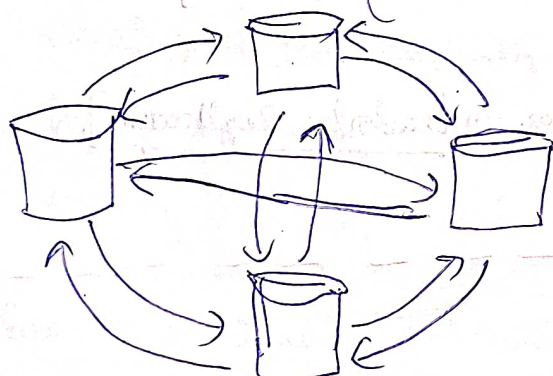


(a) circular topology



(b) star topology.

Problem with the above both topologies is that if 1 node fails, it can ~~also~~ interrupt the flow of replication messages between other nodes causing them unable to communicate until the node is fixed → (Single Point of Failure)



(c) All-to-All topology

⇒ All to all is safer in terms of Single point of Failure. No such Single point of failure.

To prevent infinite replication loops, each node is given a unique identifier, and in the replication log of each node it can check if the message has been logged or not. Each write is tagged with identifiers of all the nodes it has passed through.

For conflict resolution $\rightarrow$ In convergent way we saw that attaching timestamp to mark Last Write Wins does not always work. Because clocks of different nodes might not be in sync.

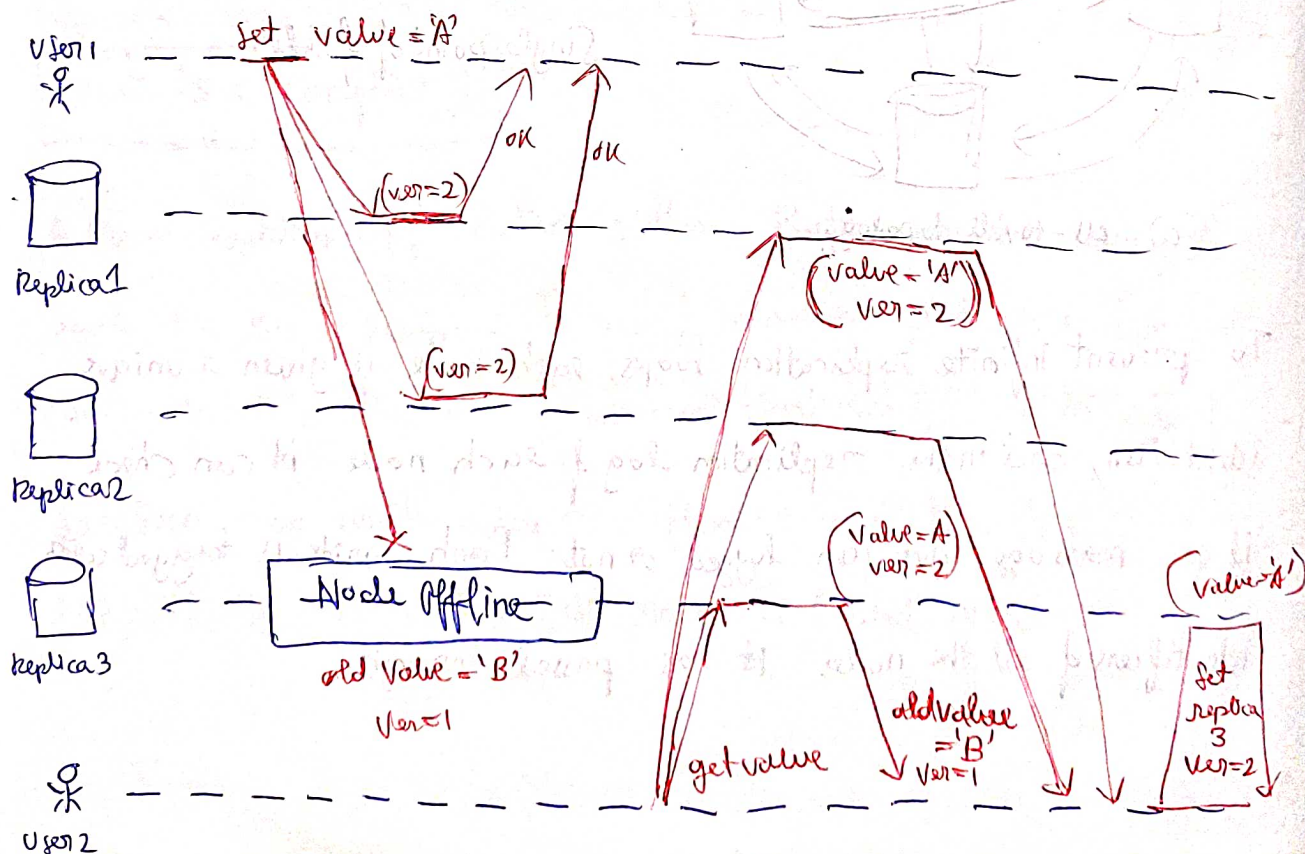
Version vectors is a technique we will learn which will help us understand more on conflict resolution.

Leaderless Replication

$\rightarrow$ In leaderless replication client sends writes to multiple clients and based on the response from multiple clients it decides on a value.

[Amazon's DynamoDB, Cassandra $\rightarrow$ Leaderless Replication]

[Let's see an example of node outage in Leaderless Replication]



these kind of read requests that sent to multiple replicas and in the end as we saw, versions are used to resolve conflicts.

Q1 How does a node which was offline and came back live after sometime read the data or get latest sync?

A1 Amazon's dynamo DB uses two methods

1) Read Repair: During read just check which replica has stale value [we query multiple replicas] And then update it with the latest version as we see in the previous figure

2) Anti Entropy process :- In some databases a background thread runs that constantly looks for differences in data between replicas and copies any missing data from one replica to another.

Quorums for reading and writing

if there are n replicas in total, every write must be confirmed by w nodes to be considered successful.

For each read query at least r nodes for each read.

As long as $w+r > n$ we can expect up-to-date value when reading. Reads and writes which obey these r and w values are called quorum reads and writes.

In some workloads $w=n$ and $r=1$, In this case read is very fast but if even 1 node fails all writes will fail to DB.

⇒ In many databases there may be more than n nodes in the cluster, but a particular partition may be stored in only m of those nodes, So for reading that specific partition quorum can be applied to those m nodes itself.

The quorum condition $w+r > n$ allows the system to tolerate unavailable nodes as follows:

$w < n \rightarrow$ Still process writes if a node is unavailable

$r < n \rightarrow$ Still process reads if a node is unavailable

Read and write queries are always sent to n replicas in parallel, the parameters w and r determine how many nodes we wait for,

If less than w or r nodes are available we give error

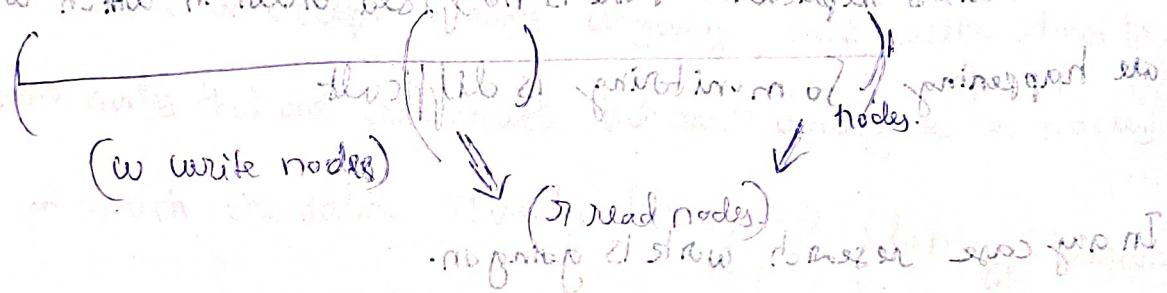
Limitations of Quorum Consistency

$(w+r \leq n)$ if this is the quorum status, then its more m

likely we will read stale values. Lower Latency ~~but~~ & higher availability. But compromise on consistency.

[Few Edge Cases where $w+r > n$ becomes unreliable]

- No overlap between the $w \rightarrow$ write nodes and $r \rightarrow$ read nodes.



- If writes occur concurrently, it is not clear which happened first.
- Read and write happen concurrently, in this case we are not sure whether we are reading old/new value.
- If writes succeeded on some replicas and not on all the w nodes, then the changes are not rolled back.

- If a node carrying a new value fails before replicating to others and when it comes back up and reads, the quorum w nodes are read which have old value break the quorum consistency.

- Nodes which have majority old values might get picked.

(Sloppy Quorums and Hinted Handoff.)

(Monitoring Staleness.) {

- * Application should alert user if replication is falling behind by a lot
- * For leader-based replication, the database exposes metrics, for monitoring Replication Lag for each follower.
- * In leaderless replication there is no fixed order in which writes are happening. So monitoring is difficult

In any case research work is going on.

}

Databases with appropriately configured quorums can tolerate the failure of individual nodes without the need for a failover.

They can tolerate a few nodes going down and don't wait for all n nodes to respond.

However → A small network interruption can cut off contact with a lot of nodes leading to a request never ~~reaching~~ reaching quorum and all requests start failing.

... being the system's ability to perform work during a split.

(*)

In a large cluster (with significantly more than n nodes) it's likely that the client can connect to some database nodes during the network interruption, just not to the nodes that it needs to assemble a quorum for a particular value.

(Important Question)

* Is it better to return errors to all requests for which we cannot reach a quorum of w or r nodes

* Or should we accept writes anyway, and write them to some nodes that are reachable but aren't among the n nodes

on which the value usually lives, $\Rightarrow$ (Sloppy Quorum)

[Sloppy Quorum]: Writes and Reads still require w and r successful responses, but these may include nodes that

are not among the designated n "home" nodes.

Analogy: If you lock yourself out of the house, you can sit in your neighbor's house for a while.

[Hinted Hand Off]

After network is resumed, writes are made to home nodes asynchronously.

Sloppy Quorums increase write availability, but read durability is compromised, as older value might be returned, because data is yet to be updated in most nodes.

As long as nodes are available we will write, might be outside of n nodes.

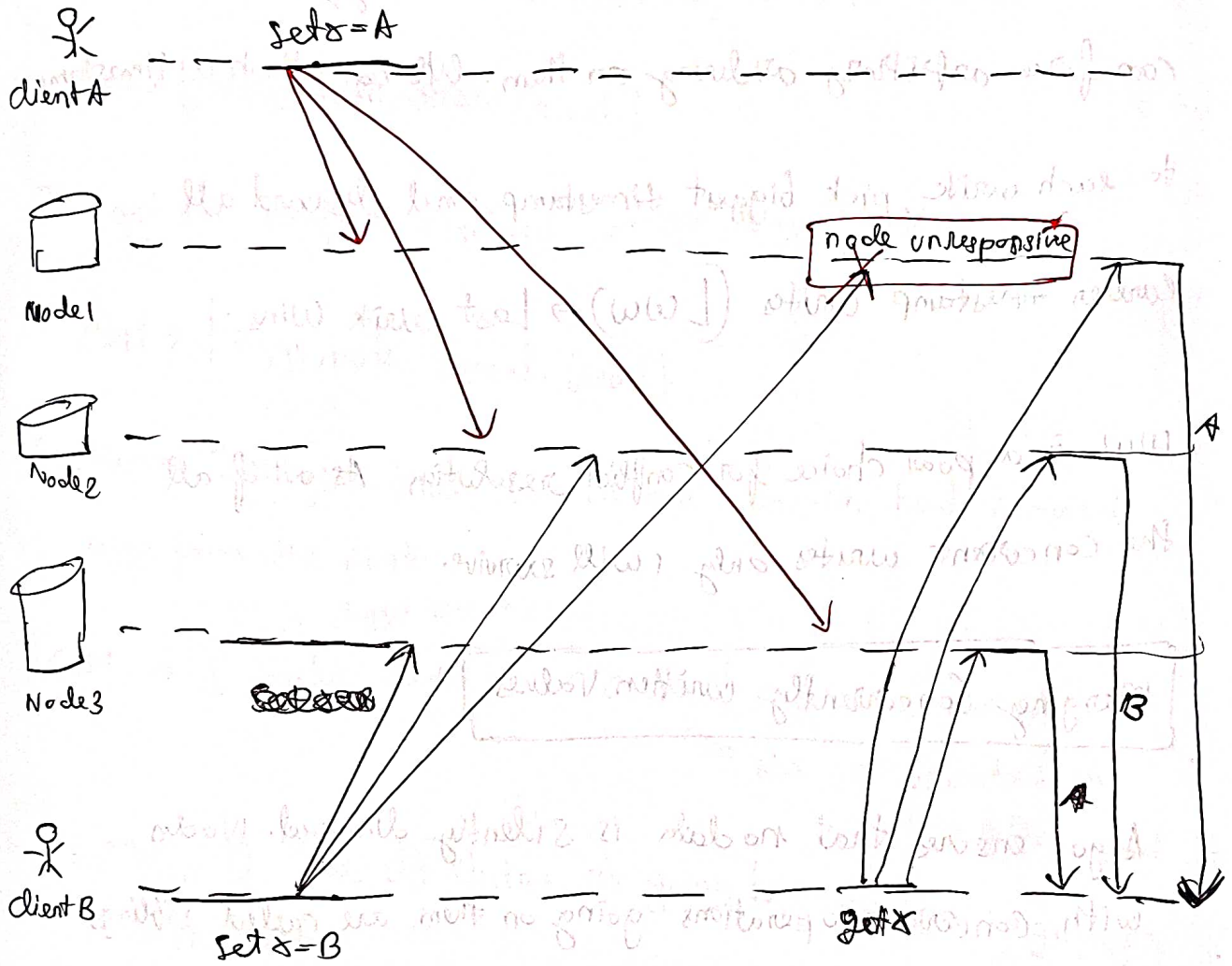
[Cassandra by default has turned off sloppy quorum]

Multi-Data Center Operation

Cassandra is multi-data center with leaderless replication, the number of replicas n includes nodes in all datacenters and in the configuration you can specify how many of the n replicas you want to have in each datacenter. Each write from a client is sent to all replicas, but the client only waits for ack from a quorum of nodes within the local data centre (whichever data centre the request had gone to). Local datacentre to avoid network and I/O calls.

Detecting Concurrent Writes

Dynama-Style Databases face the issued of Concurrent writes



✗✗ If we simply overwrite then responses become inconsistent as we see in the figure

Developer needs to know in detail about database internals to handle conflicts.

- ①
- ②
- ③

Last write wins (discarding concurrent writes).

[Convergent way]

Even though we don't have natural ordering, we can force arbitrary ordering on them. We can attach a timestamp to each write, pick biggest timestamp and discard all earlier timestamp writes (LWW) $\rightarrow$ Last write wins.

LWW is a poor choice for conflict resolution. As out of all the concurrent writes only 1 will survive.

Merging Concurrently written values

Algo ensures that no data is silently dropped. Nodes with concurrent operations going on them are called siblings.

Merging siblings is like resolving github conflict.

When there is a Conflict when we pull from upstream,

we are given 3 choices.

① Accept theirs

② Accept Yours

③ Union Merge

In union merge we take union of both the nodes.

Let's say that we have add to cart feature.

Some replica has cart_1 [eggs, milk, bread]

cart_2 [eggs, milk, ~~bread~~ cereal].

To get the conflict resolved we are doing $[\text{cart}_1 \cup \text{cart}_2]$

$\text{cart} \rightarrow$ [eggs, milk, cereal, bread]

Problem $\times$ Imagine that user in one operation had removed eggs from the cart.

$\rightarrow$ eggs removed.

$\text{cart}_1 \rightarrow$ [milk, bread]

$\text{cart}_2 \rightarrow$ [eggs, milk, cereal]

$\Rightarrow$ Not yet updated in

replica.

Then if we do Union it's going to add eggs again to the cart.

Hence we have a marker tombstone.

$\rightarrow$ tombstone

$\text{cart}_1 \rightarrow$ [eggs(T), milk, bread]

$\text{cart}_2 \rightarrow$ [eggs, milk, cereal].

Union $\rightarrow$ [egg marked as tombstone

$\text{cart} \rightarrow$ [milk, bread, cereal].

So it is deleted]

"Happens-before" Relationship and Concurrency

A operation 'A' happens before another operation 'B' if B knows about A, or depends on A, or builds up on A in some way.

Whether an operation happens before ~~the~~ another operation is the key to defining what concurrency means.

In fact we can simply say two operations are concurrent if neither knows about the other.

Whenever 2 operations are happening, there are 3 options.

- 1) A happened before B
- 2) B happened before A
- 3) Both are concurrent.

We need an algorithm to tell us if two operations are concurrent or not.

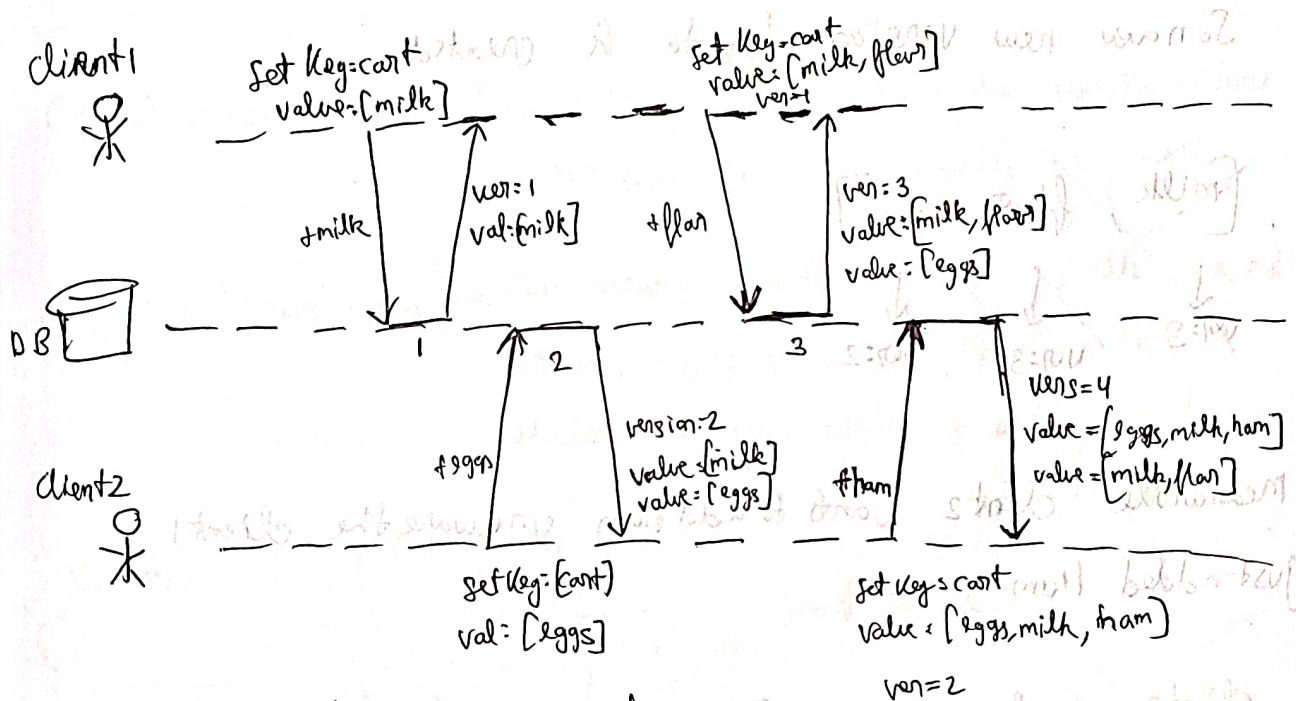
If we know one operation happens before the another, then we should simply overwrite it.

If (concurrent), there is a conflict we need to resolve.

Concurrency literally doesn't mean the events occurred at the same time. Rather the replica doesn't know which orders the event occurred.

Capturing the happens before Algorithm

Let's look at an algorithm which tells if an operation happened before another or were the operation is concurrent.



Now let's see what is happening here.

1. Client1 adds milk to the cart. Database returns it [milk] version=1
2. Client2 adds eggs agnostic of the change made by client1. and when it reaches to database, it sees milk was already there in the cart. So it does not modify the snapshot version 1 of the cart and make it [milk, eggs] version 2.

3. Client 1, oblivious to client 2's write, wants to add flour to cart,

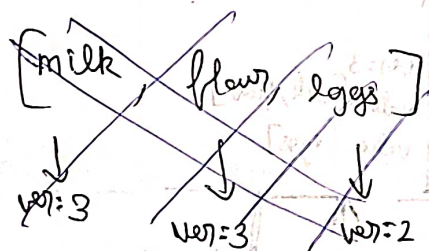
so it makes $\text{cart} = [\text{milk}, \text{flour}]$ version = 1,

It sends the value to server, along with the version.

The server sees that it has $[\text{milk of version 2}]$ added already.

It has that snapshot already.

So new new version has to be created



4. Meanwhile client 2 wants to add Ham unaware the client 1 just added Ham.

Client 2 sends $\rightarrow \text{cart} = [\text{eggs}, \text{milk}, \text{ham}]$
version = 2

~~Server sees that~~

Response $\rightarrow [\text{eggs}, \text{milk}, \text{ham}] \rightarrow \text{version 4.}$

$[\text{milk}, \text{flour}] \rightarrow \text{version 3}$

Now let's break down what is happening Actually

So we can imagine that version is like a snapshot only.
For a version/snapshot newly generated, DB maintains a copy.

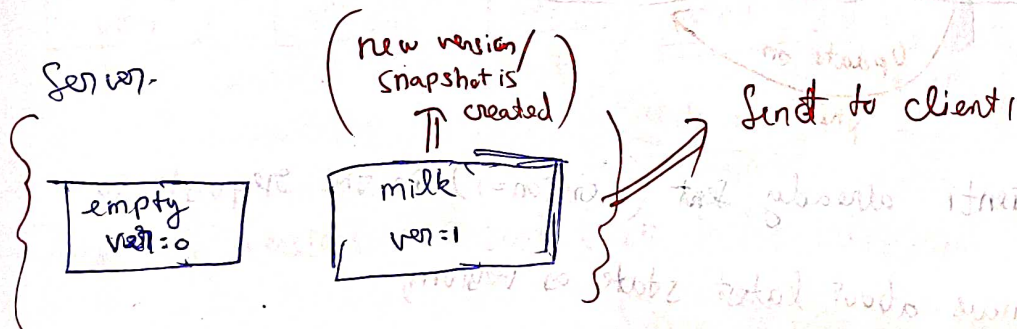
(Initially Cart Empty)

Cart → Empty Version 0 → We don't maintain a version 0 snapshot in the DB, it is just for explanation.

(Step 1)

Client adds milk: Does not explicitly send the version 0 here but let's say for understanding it sends

[milk] ~~ver=0~~ → this means client is adding milk based on version 0's snapshot. ~~He~~ From him version 0 is what he is updating on.



(Step 2)

client2 adds eggs: Does not know anything about milk being added so it also sends version 0.

Client 2 → ~~ver=0~~ [eggs] ver=0 → Client 2 wants to update cart assuming version 0 is the latest version.



(Step 3) Client1 wants to add flour to the cart

Last operation in Step 1, if we remember client added milk
he got response [client1] ver:1

So Now

[milk, flour] = ver 1 → request to cart

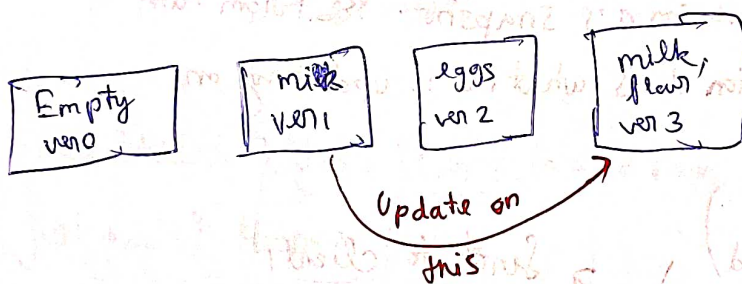
Client1 is sending ver 1,

Basically saying am making this update assuming

ver 1 → milk is the latest cart status

Server

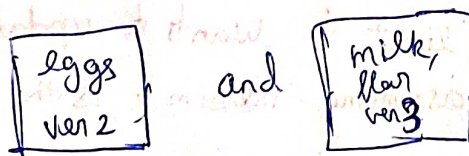
Receives the [milk, flour] ver 1



Since the client1 already sent (version=1) in the request, so it already knows about latest state as version 1,

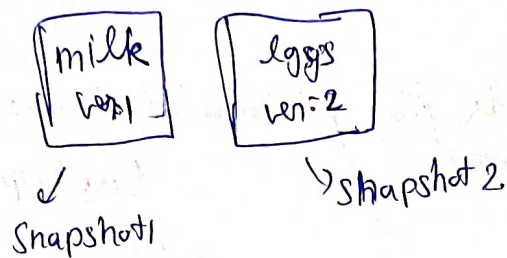
So we need to send updates to it happened after it.

So we send it,



Step 4) Client 2 wants to Ham to cart.

The last response client 2 has gotten from server is

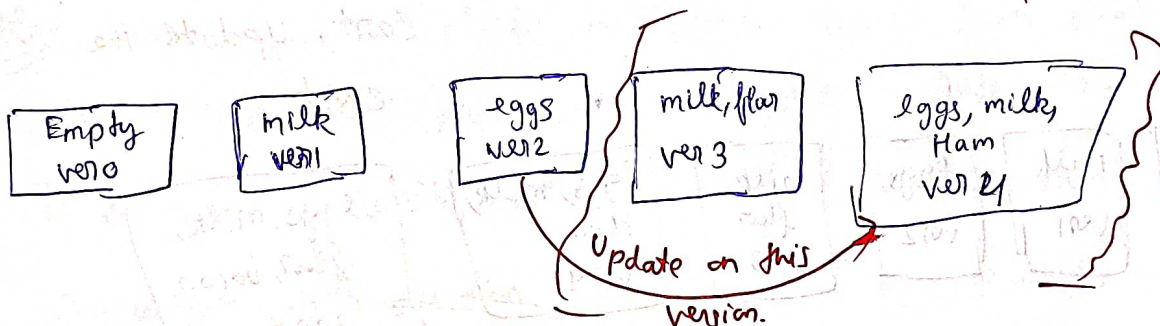


So client 2 now knows that snapshot 2 as latest current cart version.

So now it merges [milk, eggs] $\Rightarrow$ assuming its part of cart

Client 2 wants to add Ham.

[milk, eggs, Ham] ver 2 $\rightarrow$ Assuming ver 2 snapshot is the latest cart picture send the request:



These two versions are sent back in the response

Since it already knows

[milk, flour] $\rightarrow$ ver 3

about ver 2:

[eggs, milk, Ham] $\rightarrow$ ver 4

(Step 5) Client₁ wants to add bacon,

Last update received by client₁ was in ("Step 3")

eggs
ver 2

milk, flour
ver 3.

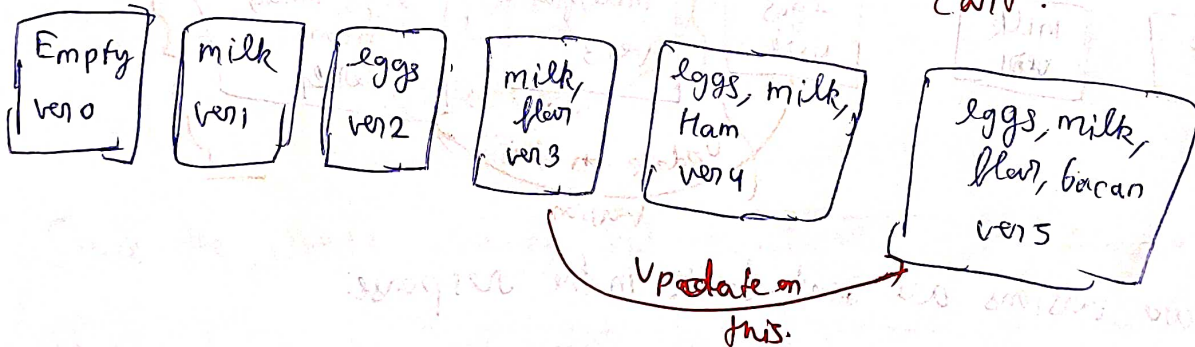
It knows about
last update of
flour so it updates
~~flour~~. ver = 3.

It merges [eggs, milk, flour] (ver = 3)

&

Request to server . [eggs, milk, flour, bacon] version 3

Assuming version 3 as
the current picture of
cart, update the
cart.



Server Response:

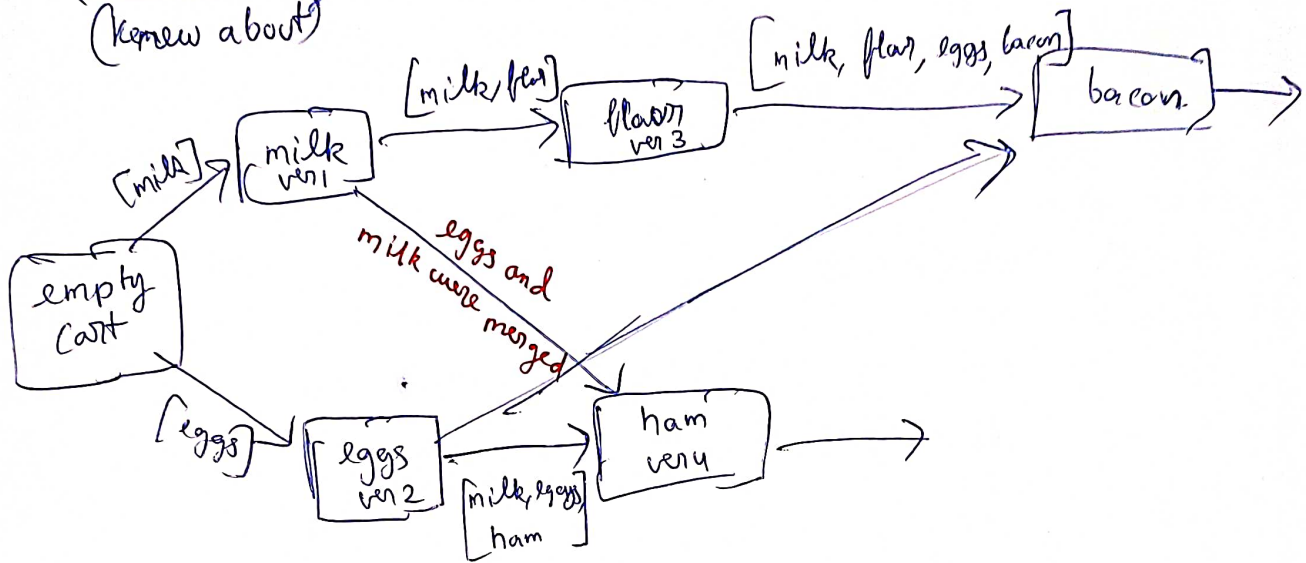
eggs, milk,
Ham
ver 4

eggs, milk,
flour, bacon
ver 5

Now using these version numbers we can draw a graph of casual dependencies.

(happend before)

(knew about)



Using this dependency graph we can tell if any operation happened before the other or the operations were concurrent.

Algo When a write includes the version number from prior read, that tells us which previous state the write is based on.

If you make the write without including a version number, it is concurrent with all other writes, so it will not even write, it will return the subsequent reads.

[Version Vectors]

↳ The above example that we saw was for a single replica to handle this across multiple replicas, we do it using version vector.

[DB₁, versions] → version vectors used to decide conflicts
 [DB₂, versions]